

TechJobs — AI-Powered Job Board Platform

A full-stack job board built with **Next.js 16** and **Neon (serverless PostgreSQL)**, featuring AI-assisted resume matching, a command palette, market insights dashboard, and company intelligence cards. Deployed on **Vercel** with a **GitHub Actions CI/CD pipeline**.

- **Live Demo:** <https://job-board-tsuki.vercel.app>
- **Repository:** <https://github.com/nikhilmangali1/job-board>

Table of Contents

1. [Tech Stack](#)
2. [Features Overview](#)
3. [Architecture](#)
4. [Installation & Setup](#)
5. [Deployment](#)
6. [API Reference](#)
7. [Project Structure](#)
8. [Design System](#)
9. [Accessibility](#)
10. [Security & Privacy](#)
11. [CI/CD Pipeline](#)

Tech Stack

Technology	Purpose
Next.js 16 (App Router)	Full-stack framework — React frontend + API routes
React 19	UI library
TypeScript	Type safety across the entire codebase

Tailwind CSS v4	Utility-first styling, responsive design, dark mode
Neon (Serverless PostgreSQL)	Primary database with tagged-template SQL queries
Vercel	Hosting and production deployment
GitHub Actions	CI/CD pipeline — lint, build, and deploy

Features Overview

1. Job Browsing & Search

The homepage presents job listings as rich cards with title, company, location, type badge, salary range, and a 2-line description preview.

- **Real-time search** across title, company, and location with 300ms debounce.
- **Filters:** Job type (Full-time, Part-time, Contract, Remote, Internship), location, minimum salary, remote-only toggle, and company name.
- **Sort options:** Newest, Oldest, Salary (high-to-low), and Best Match (when a resume is loaded).
- **Shareable state:** All active filters and sort order are encoded in URL query parameters, so a filtered view can be shared via a plain link.
- **Popular search tags:** Quick-access chips for common searches like "React", "Python", "Remote", "Bangalore".
- **Skeleton loading** while data fetches, and an empty state when no jobs match.
- **Active filter chips** with inline removal buttons.

2. Job Details Page

A dedicated page for each job at `/jobs/[id]` featuring:

- Full description, requirements, company, location, type, salary, and posted date.
- **Apply Now** button linking to an external application URL when provided.
- **Resume Match** badge and expandable match breakdown when a resume has been analyzed.
- **Company Hover Card** — hover the company name to see hiring activity, recent job count, average salary, technologies used, and locations.
- Automatically tracked as a "Recently Viewed" job on page load.

3. Post a Job

The job posting form at `/post` provides a guided publishing experience:

- **8 fields:** Title, Company, Location, Type, Salary Range, Description, Requirements, Apply URL.
- **Completion progress bar** with real-time percentage tracking.
- **Job type selector** — pill buttons for Full-time, Part-time, Contract, Remote, Internship.
- **Smart suggestions** — dynamic tips based on form content (title length, salary inclusion, description detail, URL validity).
- **Live preview** in the sidebar showing how the listing will render.
- **Character counter** and word count for the description field (max 1000 characters).
- **Client-side validation** with inline error messages and `aria-invalid` attributes.
- **Server-side validation** on the API route as a secondary check.
- **Draft auto-save** to `localStorage` — an in-progress post survives browser refresh.
- **Preview modal** with focus trapping, escape-to-close, and body scroll lock.

4. AI-Assisted Resume Matching

Entirely client-side — no resume data is sent to any server.

- **Input methods:** Paste text directly or upload a `.txt` file.
- **Skill dictionary:** 121 skills across 7 categories — Languages (21), Frameworks (26), Databases (16), Cloud (14), DevOps (16), Testing (11), Web (17).
- **Extraction:** `extractSkills()` uses case-insensitive substring matching against the dictionary.
- **Scoring:** `computeMatch()` applies a weighted algorithm:
 - Title skills — **3 × weight**
 - Requirements skills — **2 × weight**
 - Description skills — **1 × weight**
- **Color-coded badge:** Green $\geq 80\%$, Amber $\geq 50\%$, Red $< 50\%$.
- **Match details:** Expandable panel listing matched vs. missing skills with individual chips.
- **Best Match sort** — surfaces the most relevant jobs first.
- **80%+ filter** — only shows jobs with score $\geq 80\%$.
- **Privacy:** Resume text and extracted skills are stored only in `localStorage`. Nothing is uploaded to the server or any third party.

5. Command Palette (⌘K / Ctrl+K)

A keyboard-first navigation modal accessible from anywhere in the app:

- **16 commands** including: Browse Jobs, Post a Job, Toggle Dark/Light Mode, filter by Remote/Full-time/Internship/Contract, Clear Filters, Sort by Best Match/Salary/Newest, Focus Search, Clear Recently Viewed, Clear Resume, Market Insights, Analyze Resume.
- **Fuzzy search** across command labels and keywords.
- **Live job search** — type any query to search jobs with fuzzy token matching (max 8 results), showing match badges when a resume is loaded.
- **Keyboard navigation** — Arrow keys to move, Enter to select, Escape to close, with auto-scroll into view.
- **Recently used** commands are tracked (max 5) and surfaced at the top.
- **Rich job results** with copy-link and save-to-localStorage actions.
- **ARIA combobox/dialog pattern** with proper roles and focus management.

6. Recently Viewed Jobs

- The last 10 jobs a visitor opens are tracked client-side and shown as a horizontally scrollable strip on the homepage.
- Each entry shows company initials, title, company, location, match badge, job type badge, and relative viewed time.
- Persisted in localStorage; clearable from the UI or command palette.

7. Market Insights Dashboard (/analytics)

A server-rendered analytics page with computed market intelligence:

- **Stats overview:** Total Jobs, Unique Companies, Remote Jobs, Average Salary, Highest Salary, Posted This Week — with animated count-up numbers.
- **Tech Trends:** Top 10 most in-demand skills across all listings (horizontal bar chart).
- **Salary Distribution:** Bucketed salary ranges (<5 LPA, 5–10 LPA, 10–20 LPA, 20+ LPA) as a bar chart.
- **Location Heatmap:** Top 8 locations by job concentration.
- **AI Insights:** Natural-language summaries covering market activity, remote work trends, hiring types, company diversity, and salary outlook.
- **Hiring Insights cards:** Most common job type, top hiring city, top company, average salary, remote work percentage.

8. Company Hover Cards

Hover any company name to see a rich popover with:

- Company avatar (deterministic gradient from company name hash) and name.
- **Hiring activity level** — Very Active (≥ 5 jobs), Active (≥ 2), Occasional (< 2), with a green/blue/yellow status dot.
- **Stats**: Total open roles, average salary, unique locations, most common job type.
- **Technologies**: Top 5 skills extracted from the company's job listings.
- **Remote label**: Remote-friendly / Some remote / On-site based on job data.
- **Smart positioning** — appears below or above the trigger based on viewport space, clamped within window edges.
- **150ms show delay** and **200ms hide delay** with timer management to prevent flickering.

9. Theme System

- Dark/light mode toggle persisted to localStorage and applied via class on `<html>`.
- **No flash-of-wrong-theme** — the theme class is applied synchronously on load.
- Sun/moon icons with hydration-safe rendering (SSR renders empty, client populates).
- Over 30 design tokens with separate dark mode values (see [Design System](#)).

10. REST API

Method	Endpoint	Description
GET	<code>/api/jobs</code>	List jobs with filtering and sorting
GET	<code>/api/jobs/[id]</code>	Fetch a single job by ID
POST	<code>/api/jobs</code>	Create a new job listing
GET	<code>/api/analytics/summary</code>	Aggregate stats for the homepage preview
GET	<code>/api/init</code>	Create the <code>jobs</code> table (idempotent)

Detailed API documentation is in the [API Reference](#) section.

Architecture

Rendering Strategy

The app uses Next.js App Router with a **hybrid rendering model**:

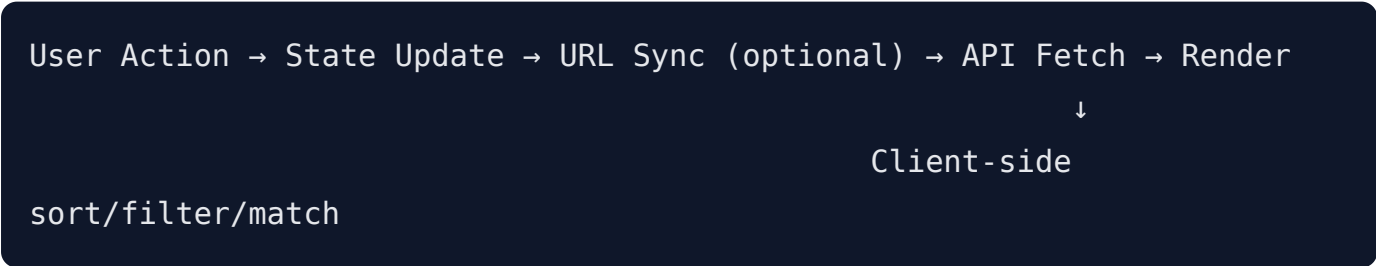
Page	Strategy	Rationale
Homepage (/)	Client-Side Rendering (CSR)	Highly interactive — filters, search, resume matching all update in real-time
Job Detail (/jobs/[id])	Client-Side Rendering (CSR)	Needs localStorage for resume data and recently-viewed tracking
Post Job (/post)	Client-Side Rendering (CSR)	Form interactions, draft autosave, live preview
Analytics (/analytics)	Server-Side Rendering (SSR, force-dynamic)	Always returns fresh data; pre-computes analytics before sending HTML

State Management

No external state library — the app uses React's built-in primitives:

- **useState / useMemo** — UI state and computed values (match scores, filtered job lists).
- **localStorage** — Persistent state for resume, theme, recently viewed, palette recent items, form drafts, and palette tip dismissal.
- **Custom event bus** (`window.dispatchEvent` / `addEventListener`) — Cross-component communication for `app:clearResume` and `app:clearRecentlyViewed`.
- **URL query params** — Shareable filter/sort state via `history.replaceState`.

Data Flow



For resume matching:

```
Paste/Upload → extractSkills() → localStorage → computeMatch(jobs) → Display Badges
```

Security

- **SQL Injection:** All database queries use Neon's tagged-template function, which parameterizes inputs automatically.
- **Resume Privacy:** Resume text never leaves the browser — no API call, no server storage.
- **Server-side validation:** The `POST /api/jobs` endpoint independently validates required fields as a defense-in-depth measure.

Installation & Setup

Prerequisites

- Node.js 20+
- npm
- A [Neon](#) PostgreSQL database

Steps

```
# Clone the repository
git clone https://github.com/nikhilmangalil/job-board.git
cd job-board

# Install dependencies
npm install

# Set up environment variables
cp .env.example .env.local
```

Edit `.env.local` with your Neon connection string:

```
POSTGRES_URL=postgresql://user:password@ep-xxxx.us-east-2.aws.neon.tech/jobboard?sslmode=require
```

```
# Start the development server
npm run dev

# Initialize the database – visit this URL once:
# http://localhost:3000/api/init
```

The app will be available at `http://localhost:3000`.

Available Scripts

Command	Description
<code>npm run dev</code>	Start development server
<code>npm run build</code>	Production build (includes TypeScript checking)
<code>npm run start</code>	Start production server
<code>npm run lint</code>	Run ESLint checks

Deployment

CI/CD Pipeline

The project uses **GitHub Actions** for continuous integration and deployment:

1. On every push/PR to `main`:

- `quality` job: `npm ci` → `npm run lint` → `npm run build`

2. On push to `main` (after quality passes):

- `deploy` job: Installs Vercel CLI → pulls production environment → builds → deploys to Vercel

Environment Variables (Production)

Set these in your GitHub repository under **Settings** → **Secrets and variables** → **Actions**:

Secret	Description
--------	-------------

POSTGRES_URL	Neon PostgreSQL connection string
VERCEL_TOKEN	Vercel API token (create at vercel.com/account/tokens)
VERCEL_ORG_ID	Vercel organization ID (from vercel link)
VERCEL_PROJECT_ID	Vercel project ID (from vercel link)

The `POSTGRES_URL` must also be set in Vercel's project environment variables for the production deployment.

API Reference

GET `/api/jobs`

Returns a list of jobs with optional filtering and sorting.

Query Parameters:

Parameter	Type	Description
<code>search</code>	string	Search across title, company, and location (case-insensitive)
<code>type</code>	string	Filter by job type (Full-time, Part-time, Contract, Remote, Internship)
<code>location</code>	string	Filter by location (case-insensitive partial match)
<code>minSalary</code>	number	Minimum salary in LPA (extracts first number from <code>salary_range</code>)
<code>remote</code>	boolean	If <code>true</code> , only show jobs with type "Remote"
<code>company</code>	string	Filter by company name (case-insensitive partial match)
<code>sort</code>	string	Sort order: <code>newest</code> (default), <code>oldest</code> , <code>salary</code>

Response: `200 OK`

```
[
  {
    "id": 1,
    "title": "Senior Frontend Engineer",
    "company": "Google",
    "location": "Bangalore",
    "type": "Full-time",
    "salary_range": "25-30 LPA",
    "description": "...",
    "requirements": "...",
    "apply_url": "https://...",
    "created_at": "2026-07-01T12:00:00.000Z"
  }
]
```

GET /api/jobs/[id]

Returns a single job by ID.

Response: 200 OK or 404 Not Found

POST /api/jobs

Creates a new job listing.

Request Body:

```
{
  "title": "Senior Frontend Engineer",
  "company": "Google",
  "location": "Bangalore",
  "type": "Full-time",
  "salary_range": "25-30 LPA",
  "description": "We are looking for...",
  "requirements": "6+ years...",
  "apply_url": "https://careers.google.com/..."
}
```

Required fields: title, company, location, type, description

Response: 201 Created or 400 Bad Request / 500 Internal Server Error

GET /api/analytics/summary

Returns aggregate statistics computed from all jobs.

Response: 200 OK

```
{
  "totalJobs": 15,
  "uniqueCompanies": 8,
  "avgSalary": 14.2,
  "highestSalary": 30,
  "thisWeek": 0,
  "remoteCount": 3
}
```

GET /api/init

Idempotently creates the jobs table and adds missing columns. Safe to call multiple times.

Response: 200 OK or 500 Internal Server Error

Project Structure

```

job-board/
├─ app/
│   ├─ api/
│   │   ├─ init/route.ts           – Database initialization
│   │   ├─ analytics/summary/route.ts – Aggregate stats endpoint
│   │   └─ jobs/
│   │       ├─ route.ts           – List & create jobs
│   │       └─ [id]/route.ts      – Get job by ID
│   └─ analytics/page.tsx         – Market Insights Dashboard
(SSR)
│   ├─ jobs/[id]/page.tsx        – Job detail page (CSR)
│   ├─ post/page.tsx             – Job posting form (CSR)
│   └─ components/
│       └─ CommandPalette/       – ⌘K command palette with fuzzy
search
│   └─ CompanyHoverCard/         – Company hover card with
activity stats
│   └─ InsightsDashboard/        – Charts, stats cards, AI
insights
│   └─ ui/                       – Reusable primitives
│       ├─ Button.tsx
│       ├─ Card.tsx
│       ├─ Badge.tsx
│       ├─ Input.tsx
│       ├─ Textarea.tsx
│       ├─ Select.tsx
│       └─ Label.tsx
│   └─ JobCard.tsx              – Job listing card with match
display
│   └─ MatchBadge.tsx           – SVG circular match score
indicator
│   └─ MatchDetails.tsx         – Expandable matched/missing
skills
│   └─ SkillChip.tsx            – Individual skill chip
component
│   └─ ResumeAnalyzer.tsx       – Resume paste/upload UI
│   └─ RecentlyViewed.tsx      – Recently viewed jobs strip

```

		— PostJobPreview.tsx	– Live preview for job posting
form			
		— PreviewModal.tsx	– Full preview modal with focus
trap			
		— InsightsPreview.tsx	– Homepage analytics preview
		— Toast.tsx	– Toast notification
		— ScrollToTop.tsx	– Route-change scroll reset
		— SkeletonCard.tsx	– Job card skeleton loader
		— SkeletonDetail.tsx	– Detail page skeleton loader
		— ThemeProvider.tsx	– Theme context provider
		— ThemeToggle.tsx	– Dark/light mode toggle
		— NavClient.tsx	– Navigation client component
		— layout.tsx	– Root layout (server component)
		— page.tsx	– Homepage (CSR)
		— globals.css	– Design system & Tailwind
styles			
		— db/	
		— index.ts	– Neon query helper & database
init			
		— hooks/	
		— useCommandPalette.ts	– ⌘ keyboard shortcut listener
		— useKeyboardNavigation.ts	– Arrow key navigation hook
		— useRecentlyViewedJobs.ts	– Recently viewed state hook
		— useRecentCommands.ts	– Command palette recent items
		— useCompanyAnalytics.ts	– Company profile computation
		— lib/	
		— analytics.ts	– Stats, trends, salary,
location, AI insights			
		— companyAnalytics.ts	– Per-company hiring profiles
		— jobMatcher.ts	– Weighted resume-to-job
matching			
		— resumeParser.ts	– Resume persistence & React
hook			
		— skillDictionary.ts	– 111 skills across 7 categories
		— salaryParser.ts	– Salary parsing and formatting
		— fuzzySearch.ts	– Token matching and text
highlighting			

```

|   └─ jobsCache.ts           – In-memory job cache (30s TTL)
|   └─ commandRegistry.tsx    – Command definitions &
filtering
|   └─ recentStorage.ts       – Palette recent items
persistence
|   └─ recentlyViewedStorage.ts – Recently viewed persistence
|   └─ utils.ts               – relativeDate, getInitials,
JOB_TYPE_STYLES
└─ .github/workflows/
   └─ ci.yml                  – CI/CD pipeline (lint + build +
deploy)
└─ next.config.ts            – Next.js configuration
└─ tsconfig.json             – TypeScript configuration
└─ package.json              – Dependencies and scripts
└─ postcss.config.mjs        – PostCSS configuration
(Tailwind)
└─ eslint.config.mjs         – ESLint flat configuration
└─ tailwind.config.mjs       – Tailwind CSS configuration

```

Design System

The complete design system is implemented in `app/globals.css` using CSS custom properties and utility classes.

Design Tokens

Token Group	Examples
Backgrounds	<code>--color-bg</code> , <code>--color-surface</code> (dark mode: darker values)
Text	<code>--color-text-primary</code> , <code>--color-text-secondary</code> , <code>--color-text-inverse</code>
Brand	Indigo-600 (#4F46E5) primary, indigo-500 hover
Semantic	Success (emerald), Warning (amber), Error (rose) — each with text, background, and border variants

Gradients	indigo-to-violet for branding, emerald-to-teal for success
Shadows	sm, md, lg, floating (for modals)
Radii	sm (6px), md (8px), lg (12px), xl (16px), 2xl (20px)
Typography	Inter/Geist system font stack
Motion	200ms/300ms/500ms durations with ease-out

Utility Classes

Class	Purpose
<code>surface-card</code>	Card with background, border, shadow
<code>surface-elevated</code>	Elevated card with hover lift effect
<code>surface-glass</code>	Glassmorphism with backdrop blur
<code>btn</code> , <code>btn-primary</code> , <code>btn-secondary</code> , <code>btn-ghost</code> , <code>btn-danger</code> , <code>btn-sm/md/lg</code>	Button variants
<code>input</code> , <code>textarea</code> , <code>select</code>	Form controls with focus ring
<code>badge</code> , <code>badge-primary/success/warning/error</code>	Badge variants
<code>chip</code> , <code>chip-active</code>	Toggle pill buttons
<code>label</code>	Section/form label
<code>divider</code>	Horizontal rule
<code>skeleton</code>	Pulsing loading placeholder
<code>gradient-text</code>	Gradient text effect
<code>animate-fade-in</code> , <code>animate-slide-up</code> , <code>animate-scale-in</code>	Entry animations

shimmer	Shimmer loading effect
nav-floating	Floating navbar with blur
progress , progress-bar	Progress indicator

Dark Mode

Every design token has a corresponding dark mode override. Dark mode is activated by the `.dark` class on `<html>`, toggled via `ThemeProvider`. The `dark` variant in Tailwind v4 uses `@custom-variant dark (&:where(.dark, .dark *))` for seamless integration.

Accessibility

The application includes the following accessibility features:

- **Skip-to-content link** at the top of every page.
- **ARIA roles and properties:** `aria-label`, `aria-expanded`, `aria-pressed`, `aria-invalid`, `aria-required`, `aria-describedby`, `aria-selected`, `aria-live`, `role="alert"`, `role="dialog"`, `role="combobox"`.
- **Keyboard navigation:** Full arrow key + Enter support in the command palette, Escape to close modals and dialogs, Tab/Shift+Tab focus trapping in modals.
- **Focus management:** Auto-focus on command palette input, focus restoration on modal close.
- **Screen reader announcements:** Live regions for resume analysis status, toast messages, and palette results.
- **Contrast:** Semantic color tokens maintain WCAG-compliant contrast ratios in both themes.
- **Reduced motion:** Animations use `prefers-reduced-motion` friendly patterns.

Security & Privacy

SQL Injection Protection

All database queries use Neon's tagged-template function (`query ...`) which parameterizes inputs automatically, preventing SQL injection.

Resume Data Privacy

Resume text and extracted skills are stored **exclusively in the browser's localStorage**. No resume data is transmitted to any server, API endpoint, or third-party service. The application architecture guarantees this — there are no API routes that accept resume data.

Server-Side Validation

The `POST /api/jobs` endpoint validates all required fields independently on the server, providing defense-in-depth beyond client-side validation.

Environment Variables

Database credentials and deployment tokens are stored as GitHub Secrets and Vercel environment variables — never committed to the repository (`.env*` files are in `.gitignore`).

CI/CD Pipeline

The pipeline is defined in `.github/workflows/ci.yml` .

Trigger

- **Push** to `main` branch
- **Pull Request** targeting `main` branch

Jobs

1. `quality` (all pushes and PRs)

Step	Action
Checkout	<code>actions/checkout@v4</code>
Setup Node.js	Node 20 with npm cache
Install	<code>npm ci</code>
Lint	<code>npm run lint</code>
Build	<code>npm run build</code> (with <code>POSTGRES_URL</code> secret)

2. `deploy` (push to `main` only, after `quality`)

Step	Action
Checkout	<code>actions/checkout@v4</code>
Install Vercel CLI	<code>npm install --global vercel@latest</code>
Pull environment	<code>vercel pull --environment=production</code>
Build	<code>vercel build --prod</code>
Deploy	<code>vercel deploy --prebuilt --prod</code>

Secrets required: `POSTGRES_URL`, `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`.

Database Schema

The `jobs` table is created and managed by the `initDB()` function in `db/index.ts`.

```
CREATE TABLE IF NOT EXISTS jobs (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  company VARCHAR(255) NOT NULL,  
  location VARCHAR(255) NOT NULL,  
  type VARCHAR(50) NOT NULL,  
  salary_range VARCHAR(100),  
  description TEXT NOT NULL,  
  requirements TEXT,  
  apply_url VARCHAR(500),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Environment Variables

Variable	Required	Description
----------	----------	-------------

POSTGRES_URL	Yes	Neon PostgreSQL connection string
--------------	-----	-----------------------------------

Related Links

- **GitHub Repository:** <https://github.com/nikhilmangali1/job-board>
- **Live Demo:** <https://job-board-tsuki.vercel.app>
- **CI/CD Pipeline:** [GitHub Actions](#)